Data Structures:

Initial thought: 2D array.

After thinking: Yes, a 2D array is a good idea. Especially since there'll be different board types, having an underlying consistent data structure will make things easier.

Just use a value if a part of the array doesn't actually exist on the board, due to the board shape. (e.g., -1)

Values:
-1: Invalid cell (not part of the board)
0: Empty hole (no peg)
1: Occupied hole (contains a peg)

Classes:

The instructions say: "It is required to separate the user interface code and the game logic code into different classes" hence a GUI class is required. The overall instructions also say to practice OOP principles, so use enough classes when required.

Reminder: Classes generally represent things (objects) or distinct responsibilities, while methods represent actions those things perform.

Reminder: Look at Lecture 8– Basic Design Principles to design your classes.

Classes + Methods *(initial ideas)*:

Board: Stores the 2D array and manages board positions

Attributes: grid, size, board_type

get_cell(), set_cell(), initialize_board()

Game: Controls the overall game and rules

new_game(), make_move(), is_game_over()

GUI: Displays the game and handles user interaction

draw_board(), on_new_game_click()

Move: Represents a particular move

^ Notes:

- Game would contain a Board object and coordinate actions.
- The Move object could contain starting row, starting column, ending row, ending column, so Game.make_move() would only receive 1 object instead of 4 values. It is optional. Could also help with undo/redo functions.
- Also optional, not mentioned: BoardFactory() class. Its job would be to create the appropriate board configuration based on the chosen type and size. I will start with implementing the logic simply in Board(), and move it to BoardFactory() if that logic becomes complicated.

Development order:

1. Board: foundation of everything
2. Move
3. Game
4. Undo/Redo (utilized with the Move(), Game() class)
5. GUI

*Note: it would be good to briefly implement and test GUI after making the Board, to ensure it's showing up correctly.

File organization:

One file for main, a separate file for each major class, and one file for unit tests.

Board:

Milestone 1:

1. Create a board based on the chosen size and type.
2. Initialize its 2D array with the correct cell states.
3. Retrieve a cell's state using its row and column.

** include default values in the constructor

Assumptions:

- Board sizes can't be even (odd creates multiple center holes, and all instructions only mention 1 center hole.)

- Diagonal moves only apply to Hexagon board (English and Diamond typically only allow orthogonal moves)

Constraints:
- No even board sizes (explained above)
- Minimum: 5x5 (seems like a good minimum)
- Maximum: 11x11 (makes sense, and GUI gets cumbersome for anything higher)
- Only integer board sizes (obviously)

Testing Board

AC Criteria to test:
- AC 1.1 Customizing a board size
- AC 1.2 Default board size
- AC 2.1 Customizing the board type
- AC 2.2 Default board type